

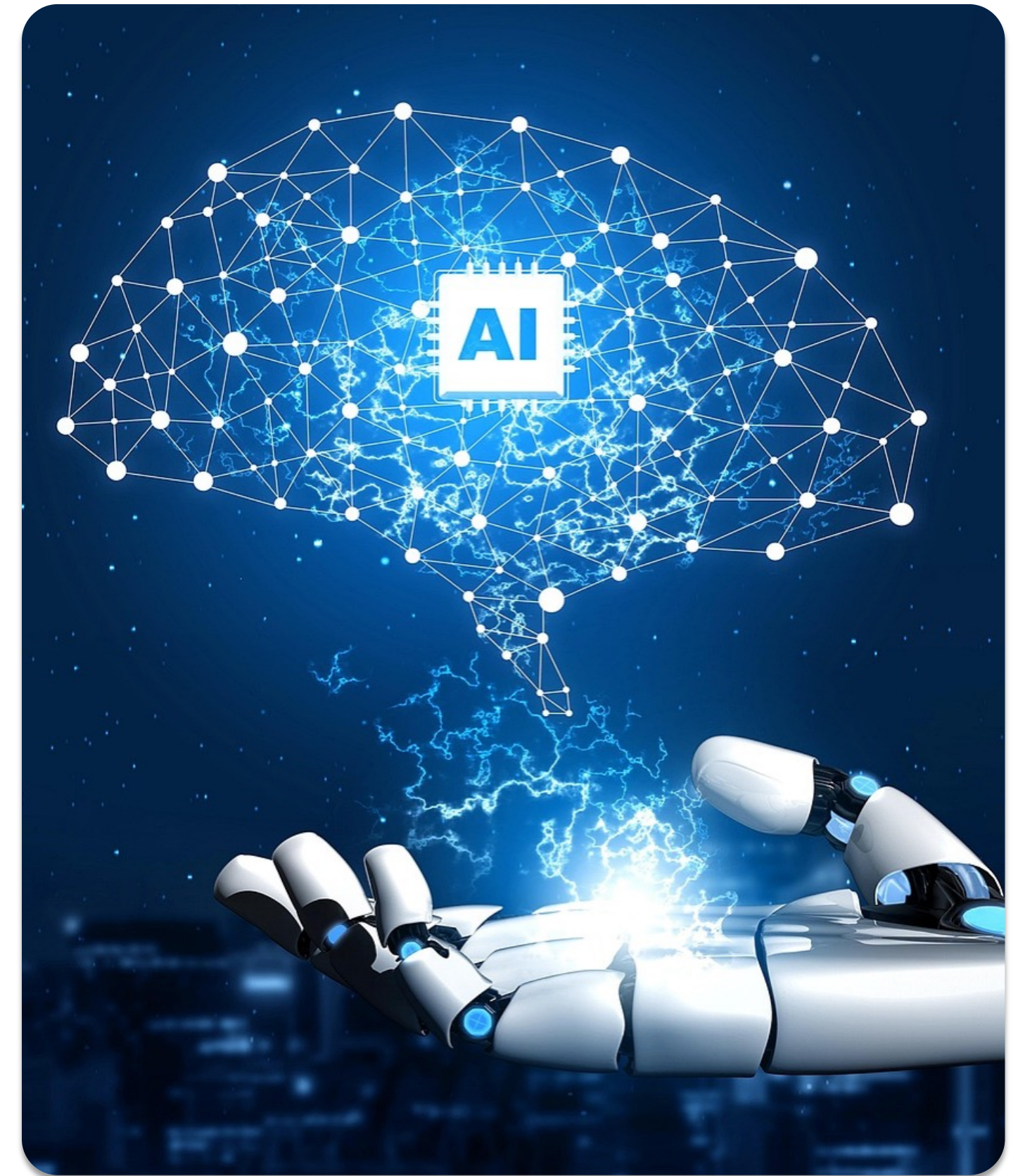
Python 기초 문법 1



텍스트, 이미지와 같은 콘텐츠(미디어)를 생성할 수 있는 인공지능

- 생성형 AI는 단순한 정보 검색을 넘어서 텍스트, 이미지, **코드**, 오디오 등 다양한 형태의 콘텐츠를 생성할 수 있음
- 기존의 자동화 기술과 다르게 새로운 결과물을 제작하는 것이 핵심

코드 또한 AI가 생성할 수 있는 결과물 중 하나



변화하는 패러다임: 공동 개발자로의 AI

기계어, 어셈블리

- 가장 저수준 (Low Level) 의 프로그래밍 언어.
- 컴퓨터의 모든 연산을 수동으로 통제.

고급 프로그래밍 언어

- 저수준 언어를 한 단계 추상화 한 고수준 (High Level) 언어.
- 한 단계 추상화를 거쳐 개발자가 알고리즘 등 기능 구현의 본질에 더 집중할 수 있도록 한다.

통합 개발 환경 (IDE)

- 기능 개발을 넘어서 프로젝트를 관리하는 개발 환경
- 개발자가 전체프로젝트를 한 눈에 보고 관리할 수 있도록 하여, 문제 해결의 본질에 한 단계 더 집중하도록 한다.

생성형 AI

- AI가 반복적인 코드 작성 작업을 자동화함에 따라, 개발자는 문제 해결, 가치 창출을 위한 창의적, 전략적인 업무에 집중할 수 있다.
- 개발자의 역할이 프로그램을 구현하는 것에서 문제 해결을 위해 AI의 생성물을 감독하고 통합하는 방향으로 진화.



변화하는 패러다임: 공동 개발자로의 AI

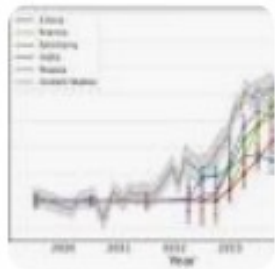
코드 생성에서 두각을 나타내는 생성형 AI

지디넷코리아

[AI는 지금] 개발 코드 30%는 AI가 작성...개발 문화도 조용히 '혁신'

개발자들이 작성하는 코드의 30%는 이미 생성형 인공지능(AI)에 의해 만들어지고 있는 것으로 나타났다. 단순한 자동완성 수준을 넘어 AI가 실질적인...

2주 전



매일경제

유니티 CEO “게임 개발자 96%가 AI 쓰기 시작...게임 개발에 AI에 이젠트 활용”

매튜 브롬버그 유니티 CEO 단독 인터뷰 생성형 AI 기술 게임 개발 엔진에 통합 '나무 200그루 배치 해줘' 하면 3D로 생성 AI, 시간 많이 드는 반복작업...

2025. 4. 27.



연합뉴스

"간단 코딩은 AI가...초급·고급 개발자 채용 양극화 더 커진다"

(서울=연합뉴스) 조성미 기자 = 인공지능(AI)이 코딩 업무를 대신하면서 개발자 채용 시장에서 초급과 고급 인력 선호도의 양극화가 빨라질 것으...

1개월 전





생성형 AI

텍스트, 이미지, 코드 혹은 다른 미디어를
생성할 수 있는 인공지능

방대한 코드 데이터로부터 프로그래밍 패턴, 구문, 추상화된 로직 까지
학습하여 주어진 요구사항에 맞는 코드를 생성할 수 있다.



개발 패러다임 비교

전통적 개발

수동으로 한 줄씩 코딩

개발자가 완전한 제어권을
가짐

핵심 기술: 언어 구문, 알고리즘 지식

AI 활용 개발

AI가 코드를 제안/완성

개발자가 AI의 제안을 검토/
수정

핵심 역량: 알고리즘 지식
기반 AI 제안 검토 능력

Vibe Coding

개발자가 자연어로 AI에게
의도를 지시

AI가 주도적으로 코드 생성,
수정

핵심 역량: 프롬프트 엔지니어링,
AI 결과물 검토 능력



전통적 개발 절차 (예시)

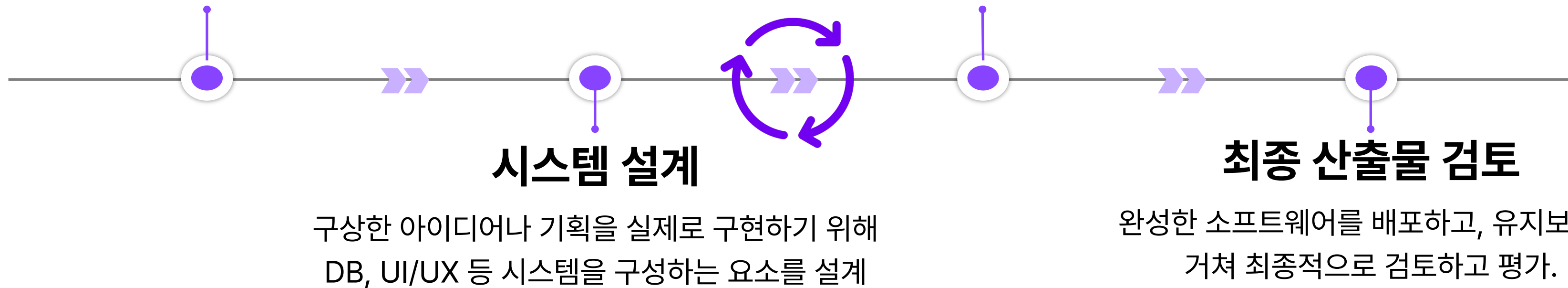
아이디어를 구상하고, 시스템을 설계하고, 코드로 구현하고, 테스트를 거쳐 배포하는 과정

아이디어 구상 (기획)

고객의 요구사항이나 주어진 문제를 해결하기 위한 핵심 아이디어나 컨셉을 구상

코드 구현, 테스트 및 디버깅

설계 명세를 바탕으로 코드를 구현하고, 다양한 테스트를 통해 올바르게 구현되었는지 확인
필요에 따라 **이전 과정으로 돌아가며 반복적으로 개선**





Vibe Coding 절차 (예시)

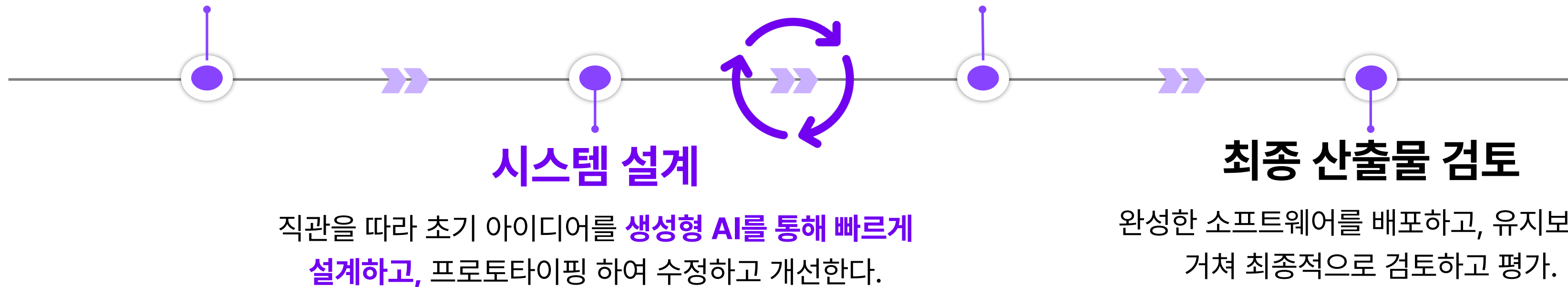
전통적 개발 절차와 흐름은 유사하지만, 각 과정을 **AI를 활용해 크게 효율화**

아이디어 구상 (기획)

고객의 요구사항이나 주어진 문제를 해결하기 위한 핵심 아이디어나 컨셉을 구상하고, **생성형 AI를 활용해 구체화 한다.**

코드 구현, 테스트 및 디버깅

생성형 AI를 활용하여 코드를 빠르게 구현하고, 다양한 테스트를 통해 올바르게 구현되었는지 확인한다. 필요에 따라 **이전 과정으로 돌아가며 반복적으로 개선**





Vibe Coding 주의점

코드의 신뢰성

- 생성형 AI는 할루시네이션 현상으로 인해 의도에 맞지 않는 코드, 틀린 코드를 생성할 수 있다.
- 최종 결정권자는 개발자 자신임을 잊지 않고, 생성된 코드를 꼼꼼하게 검수해야 한다.

보안 취약점

- AI가 학습한 코드 데이터 중 보안 면에서 취약한 코드가 있다면, 해당 취약점이 생성된 코드에도 남아 있을 수 있다.
- 코드의 신뢰성 보장과 마찬가지로, 이 역시 개발자가 꼼꼼하게 검수해야 한다.

기술 역량 퇴보 위험

- AI가 생성한 코드를 무비판적으로 수용하다 보면, AI가 생성한 코드를 읽고, 이해하고, 검수할 수 있는 역량이 퇴보할 수 있다.
- 코드를 검수하는 과정에서 코드의 의미, 구조 등이 의도와 설계에 부합하는지 확인할 수 있도록 **기술 역량을 유지하는 것이 좋다.**

Vibe Coding을 통한 생산성 향상의 한계 인지

- 작은 프로젝트에서는 한 개발자가 프로젝트를 위한 모든 요소를 혼자서 개발할 수 있지만, **복잡도가 높거나, 규모가 큰 프로젝트에서는 그렇지 않다.**
- 각자가 Vibe Coding을 통해 생산성을 어느 정도 까지 향상시킬 수 있는지 이해하는 것이 좋다.



Vibe Coding과 개발자의 역할

전통적 개발: 구현자

- 수동으로 코드 작성
- 언어, 구문, 알고리즘에 집중
- 완전한 통제권을 갖고 각 기능 구현

Vibe Coding: AI 관리자

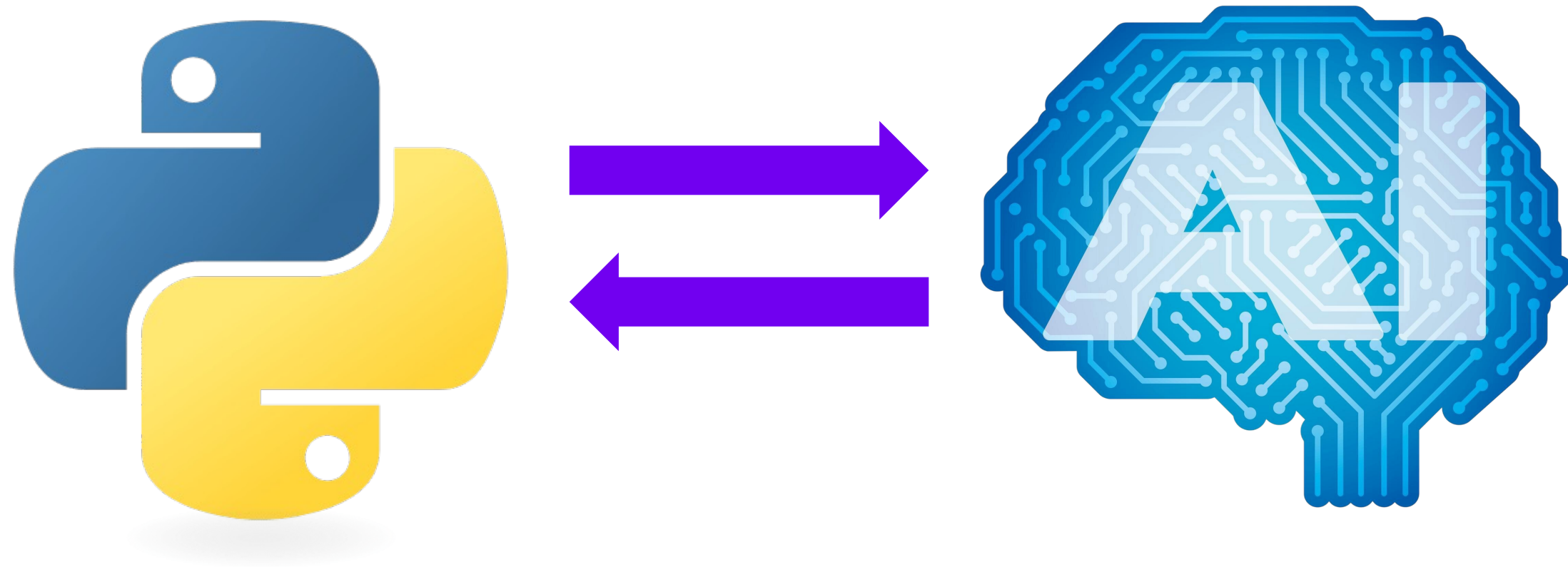
- AI에게 자연어로 의도 전달
- 시스템 구조 설계
- AI 생성물 검증/시스템 통합
- 전략적 문제 해결에 집중



AI가 Python을, Python이 AI를 더 강하게 만드는 여정

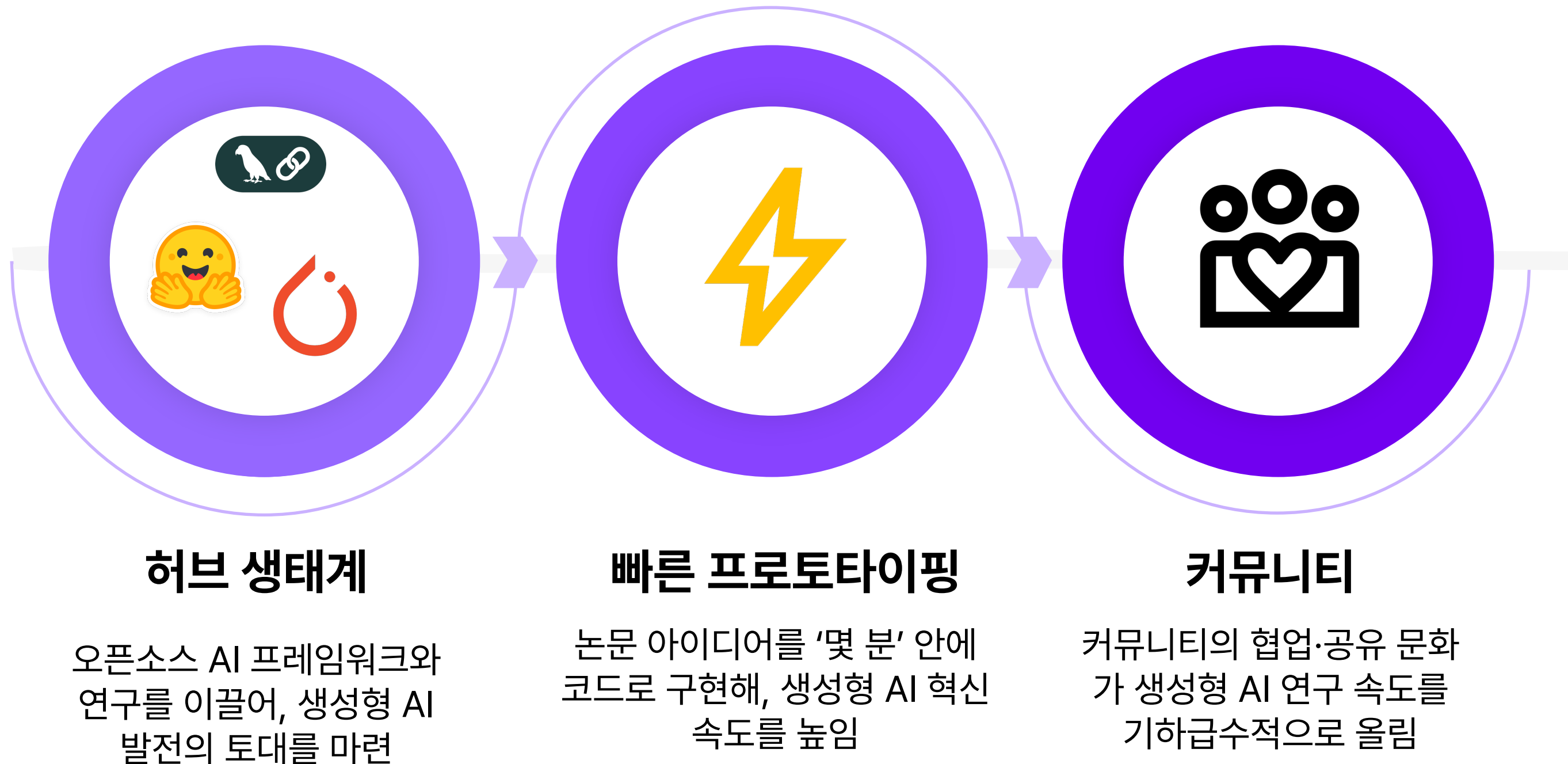
함께 성장하는 두 기술

Python이 AI 생태계를 키우고, 생성형 AI가 Python 개발 생산성을 비약적으로 높인다





생성형 AI에서 Python의 중요성





Python에서 생성형 AI의 중요성

리스트에서 중복 요소를 제거하는 코드 작성

직접 코드를 작성하는 경우

리스트에서 중복 요소를 제거하는 함수 찾기



해당 함수 사용법 문서(DOCS) 확인해보기



코드 직접 타이핑하기

VS

생성 AI를 활용해서 코드를 생성하는 경우

Python 리스트에서 중복 요소를 제거하는 코드 생성해줘

1초 동안 생각함

아래에 Python 리스트에서 중복 요소를 제거하는 대표적인 방법 세 가지를 예시와 함께 제시합니다. 필요에 따라 가장 적합한 방식을 선택해 사용하세요.

1. `dict.fromkeys()`를 이용해 순서 유지하며 중복 제거

Python 3.7+에서는 dict가 삽입 순서를 보장하므로, 이 방법으로 원본 리스트의 순서를 유지하면서 중복을 제거할 수 있습니다.

```
python
def remove_duplicates_preserve_order(seq):
    """
    입력:
        seq (list): 중복 요소를 제거할 리스트
    반환:
        list: 순서를 유지하며 중복이 제거된 리스트
    """
    return list(dict.fromkeys(seq))

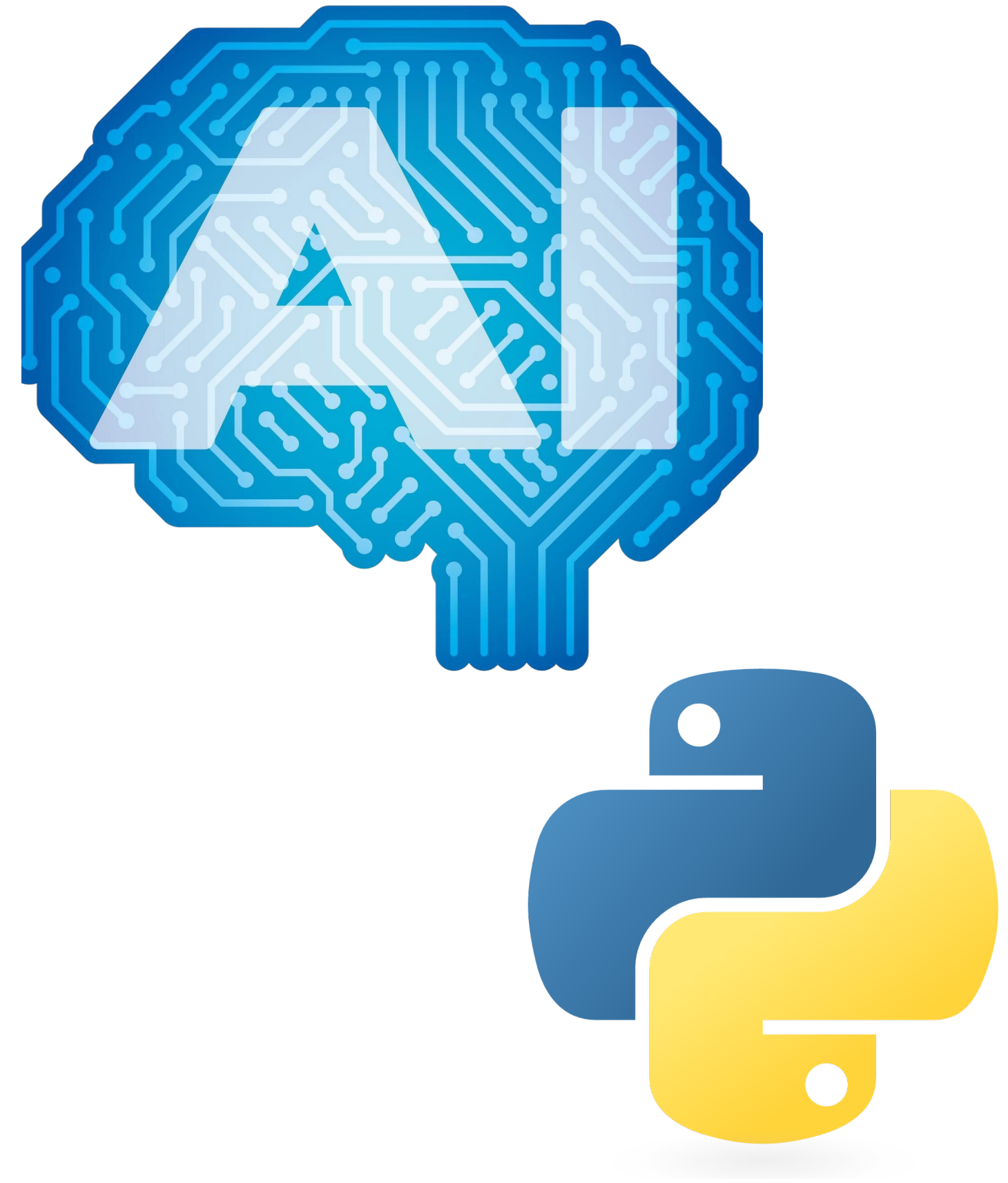
# 사용 예시
orig = [3, 1, 2, 3, 2, 4, 1]
result = remove_duplicates_preserve_order(orig)
print(result) # 출력: [3, 1, 2, 4]
```



생성형 AI는 "뇌(생각)", Python은 "손(도구)"

환경을 제어할 수 없는 생성형 AI

- 생성형 AI는 글쓰기, 요약, 코드 작성 등 창의적인 작업을 할 수 있음
- 작업자의 요청을 받아도 작업자의 환경에서 직접적으로 작업을 하기 어려움
 - 작성자 : 메일 작성 요청 → 생성형 AI : 가능
 - 작성자 : 메일 작성 후 전송 요청 → 생성형 AI : 불가능
- Python은 이러한 제약을 극복하는 "생성형 AI의 생각을 구현하는 도구"





Python과 생성형 AI의 상생

사용자의 요청

“메일 내용을 작성해 줘”

“메일 내용을 작성해서 전송해 줘”

생성형 AI의 작업

메일 내용 작성

메일 내용 작성

메일 전송 코드 작성

Python의 작업

메일 전송 코드 실행

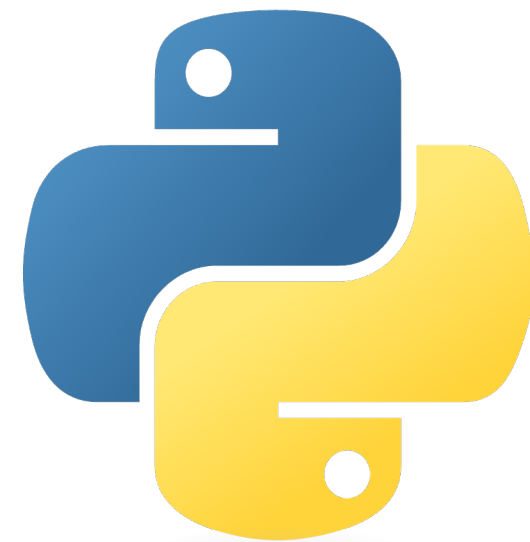


AI가 다 해주는데, 왜 아직도 Python을 배워야 할까?

코드 작성...꼭 배워야 하나..

if

for



Python

while

def

만약, AI가 만들어준 코드에서 오류가 나면?

AI가 생성한 코드에서 **뭐가 잘못됐는지 알 수 있어야** 고칠 수 있음

텍스트 파일 "example.txt"를 불러오는 코드 작성해줘

다음은 ./dataset/example.txt 파일을 불러오는 간단한 Python 코드입니다:

```
python
```

```
# 파일 경로
file_path = "./dataset/example.txt"

# 텍스트 파일 불러오기
with open(file_path, "r", encoding="utf-8") as file:
    contents = file.read()

# 내용 출력
print(contents)
```

설명:

- open() 함수로 텍스트 파일을 엽니다.
- encoding="utf-8"은 파일 인코딩을 명시하는 것으로, 한글이 포함된 파일에서도 오류 없이 읽기 위해 필요할 수 있습니다.
- with 구문은 파일을 자동으로 닫아줍니다.

필요하다면 줄 단위로 읽는 코드도 제공해드릴게요.

무엇이든 부탁하세요

```
In [1]: # 파일 경로
file_path = "./dataset/example.txt"

# 텍스트 파일 불러오기
with open(file_path, "r", encoding="utf-8") as file:
    contents = file.read()

# 내용 출력
print(contents)

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[1], line 5
      2 file_path = "./dataset/example.txt"
      4 # 텍스트 파일 불러오기
----> 5 with open(file_path, "r", encoding="utf-8") as file:
      6     contents = file.read()
      8 # 내용 출력

File ~/./local/lib/python3.10/site-packages/IPython/core/interactiveshell.py:324, in _modified_open(file, *args, **kwargs)
    317 if file in {0, 1, 2}:
    318     raise ValueError(
    319         f"IPython won't let you open fd={file} by default "
    320         "as it is likely to crash IPython. If you know what you are doing, "
    321         "you can use builtins' open."
    322     )
```

FileNotFoundError: [Errno 2] No such file or directory: './dataset/example.txt'

이름	수정일
data	오늘 오후 1:37
example.txt	오늘 오후 1:37
exercise.ipynb	오늘 오후 1:36

[에러 발생 이유]
잘못된 파일 경로

Python을 알면, 내가 원하는 대로 수정할 수 있어요

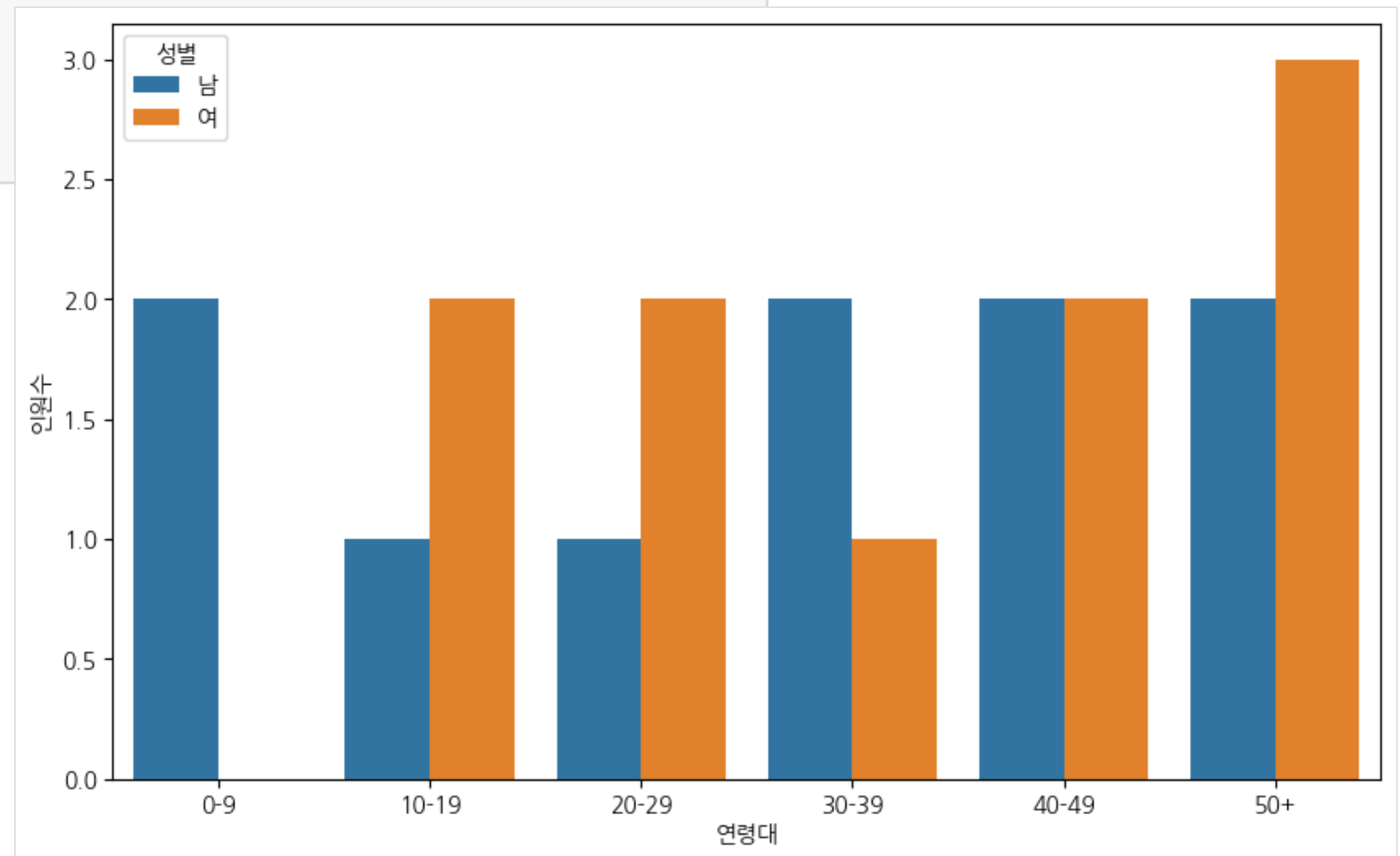
그래프의 제목을 설정하고 싶은데?

```
In [14]: # 나이 구간 정의
bins = [0, 9, 19, 29, 39, 49, float('inf')]
labels = ['0-9', '10-19', '20-29', '30-39', '40-49', '50+']

# 나이 카테고리 컬럼 추가
df['연령대'] = pd.cut(df['나이'], bins=bins, labels=labels, right=True)

# 연령대-성별 별로 집계
age_gender_counts = df.groupby(['연령대', '성별'], observed=True).size().reset_index(name='인원수')

# 시각화
plt.figure(figsize=(10, 6))
sns.barplot(data=age_gender_counts, x='연령대', y='인원수', hue='성별')
```



Python을 알면, 내가 원하는 대로 수정할 수 있어요

그래프의 제목을 설정하고 싶은데?

```
In [14]: # 나이 구간 정의
bins = [0, 9, 19, 29, 39, 49, float('inf')]
labels = ['0-9', '10-19', '20-29', '30-39', '40-49', '50+']

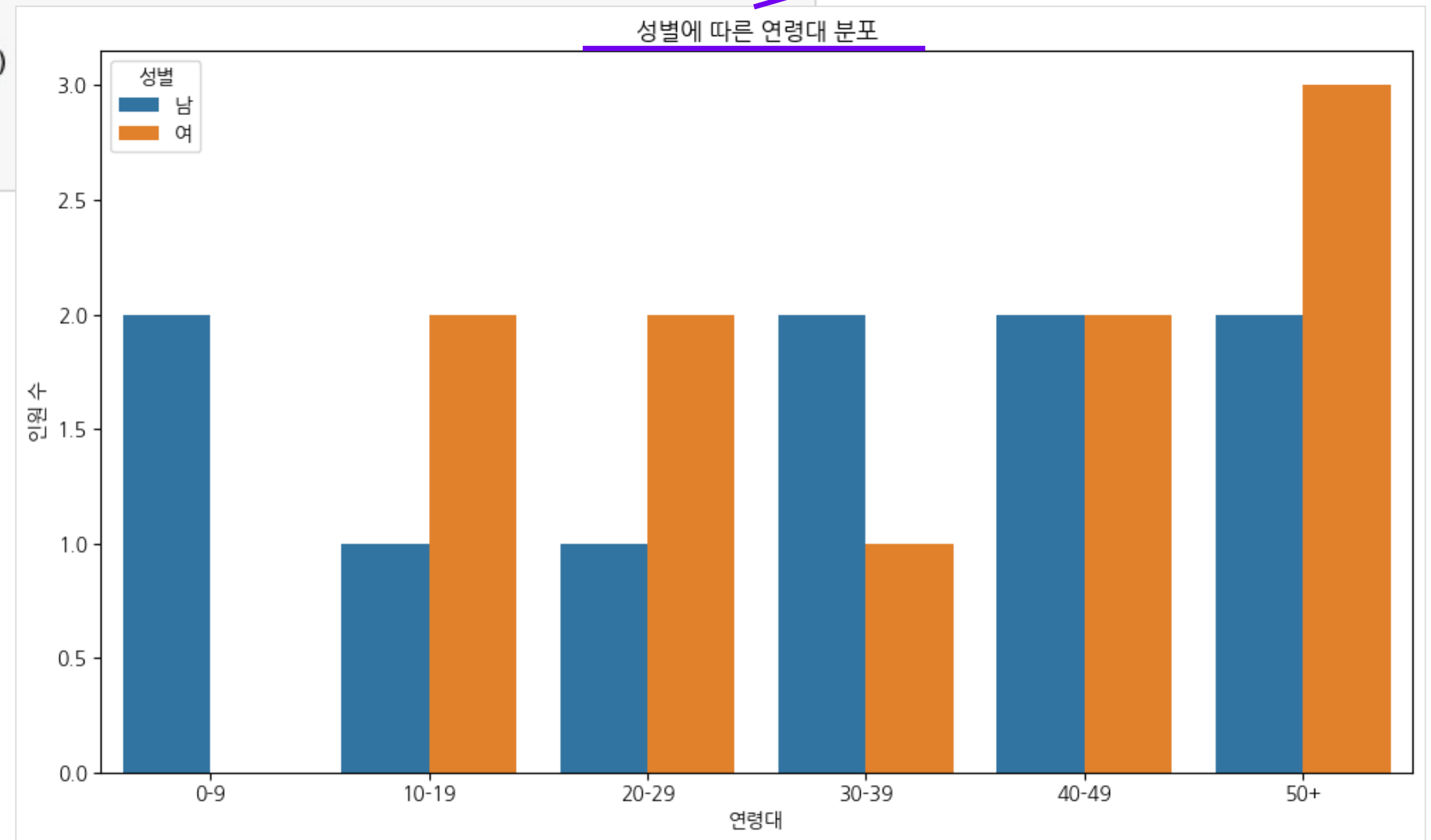
# 나이 카테고리 컬럼 추가
df['연령대'] = pd.cut(df['나이'], bins=bins, labels=labels, right=True)

# 연령대-성별 별로 집계
age_gender_counts = df.groupby(['연령대', '성별'], observed=True).size().reset_index(name='인원수')

# 시각화
plt.figure(figsize=(10, 6))
sns.barplot(data=age_gender_counts, x='연령대', y='인원수', hue='성별')

plt.title('성별에 따른 연령대 분포')
plt.show()
```

성별에 따른 연령대 분포

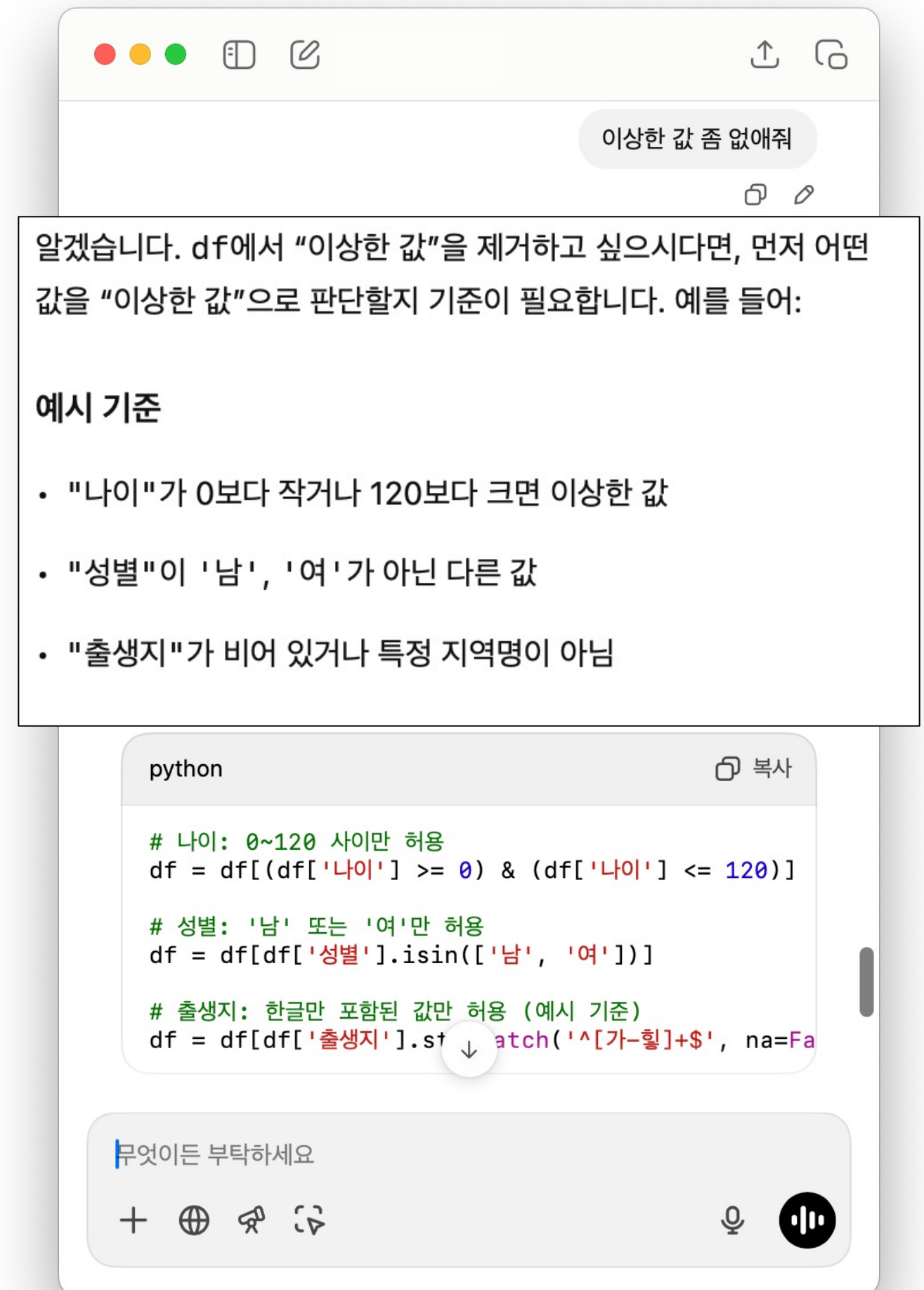


AI에게 좋은 질문을 하려면 기본 용어를 알아야 해요

[프롬프트]

이상한 값 좀 없애줘

- 생성형 AI는 사용자의 요청(Prompt)에 따라 코드 생성
- 하지만 **정확한 요청** 없이는 AI도 제대로 이해 못 함
- 예시
 - 목표 : 결측치 제거
 - 이상한 값 → 애매한 표현
 - 정확한 판단을 위해 추가 기준 필요



이상한 값 좀 없애줘

알겠습니다. df에서 "이상한 값"을 제거하고 싶으시다면, 먼저 어떤 값을 "이상한 값"으로 판단할지 기준이 필요합니다. 예를 들어:

예시 기준

- "나이"가 0보다 작거나 120보다 크면 이상한 값
- "성별"이 '남', '여'가 아닌 다른 값
- "출생지"가 비어 있거나 특정 지역명이 아님

```
python
# 나이: 0~120 사이만 허용
df = df[(df['나이'] >= 0) & (df['나이'] <= 120)]

# 성별: '남' 또는 '여'만 허용
df = df[df['성별'].isin(['남', '여'])]

# 출생지: 한글만 포함된 값만 허용 (예시 기준)
df = df[df['출생지'].str.match('^[가-힣]+$')]
```

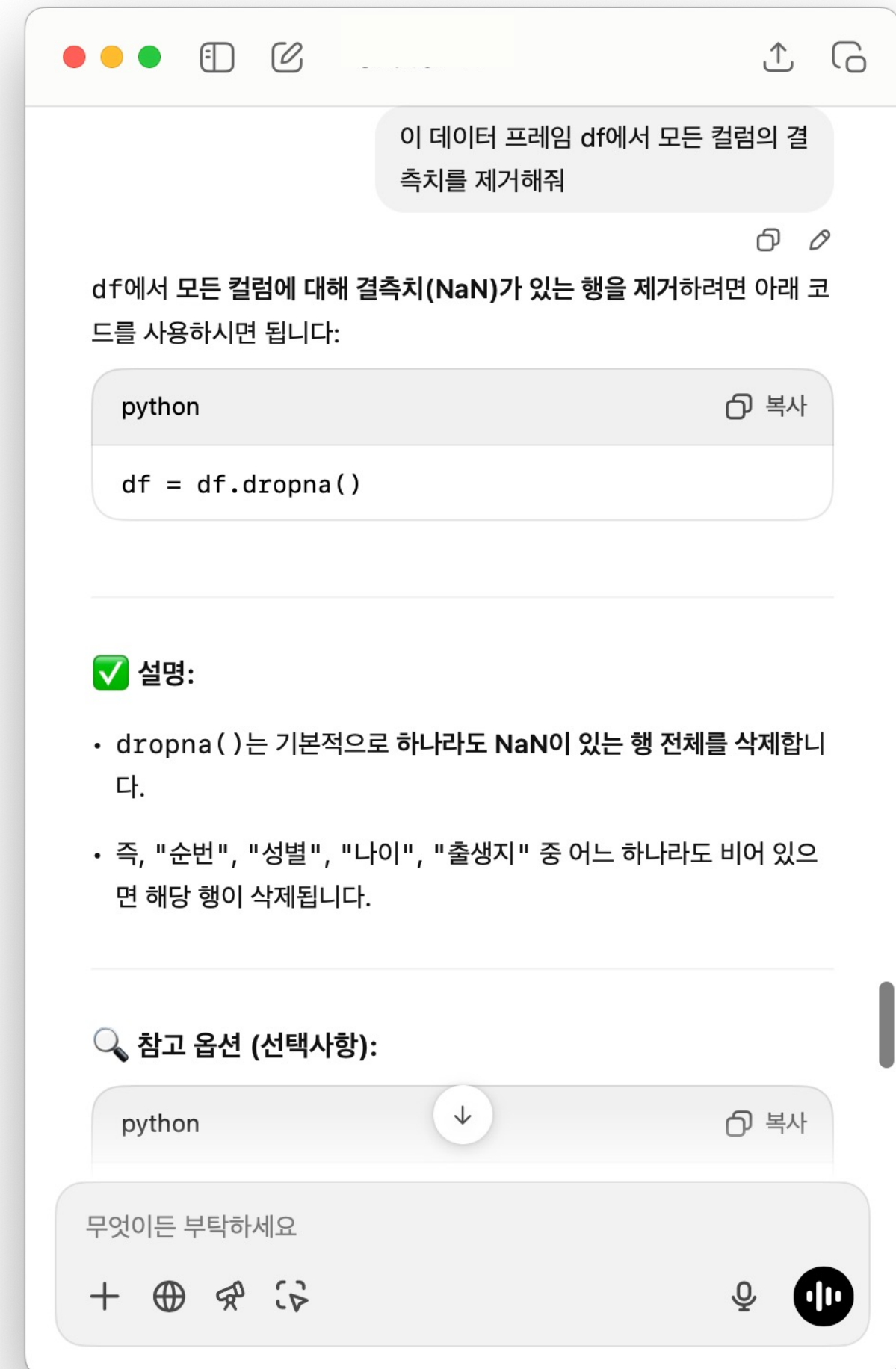
무엇이든 부탁드립니다

AI에게 좋은 질문을 하려면 기본 용어를 알아야 해요

[프롬프트]

이 데이터 프레임 df에서
모든 컬럼의 결측치를 제거해줘

- 결측치 → 명확한 처리 가능
- AI는 번역기와 같다
 - 내가 **정확한 문장**을 써야 **좋은 번역**이 나옴
 - 마찬가지로, AI에게도 **정확한 표현**이 필요



☑ 파이썬 자료형

	설명	예시
int	정수형	1, 4, 10
float	실수형	1.23, 3.08
NoneType	None	None
str	문자열	"Hello, Python!"
list	리스트	[1,2,3,4]
bool	논리형	True, False
tuple	튜플	(1,2,3,4)
dict	딕셔너리	{'a':1, 'b':2}
set	세트	{1,3,5,7}



④ 자료형 확인과 변경

자료형 확인을 위해서는 `type()` 함수를 활용합니다.

```
type(3.14) # <class 'float'>
```

자료형 변경도 가능합니다.

```
int(3.14) # 3
```



☑ 문자열 출력하기

`print()` 함수를 활용하여 원하는 문자열을 출력할 수 있습니다.

```
print("Hello, Elice!")  
print("파이썬은 재밌어!")
```

실행결과

```
Hello, Elice!  
파이썬은 재밌어!
```

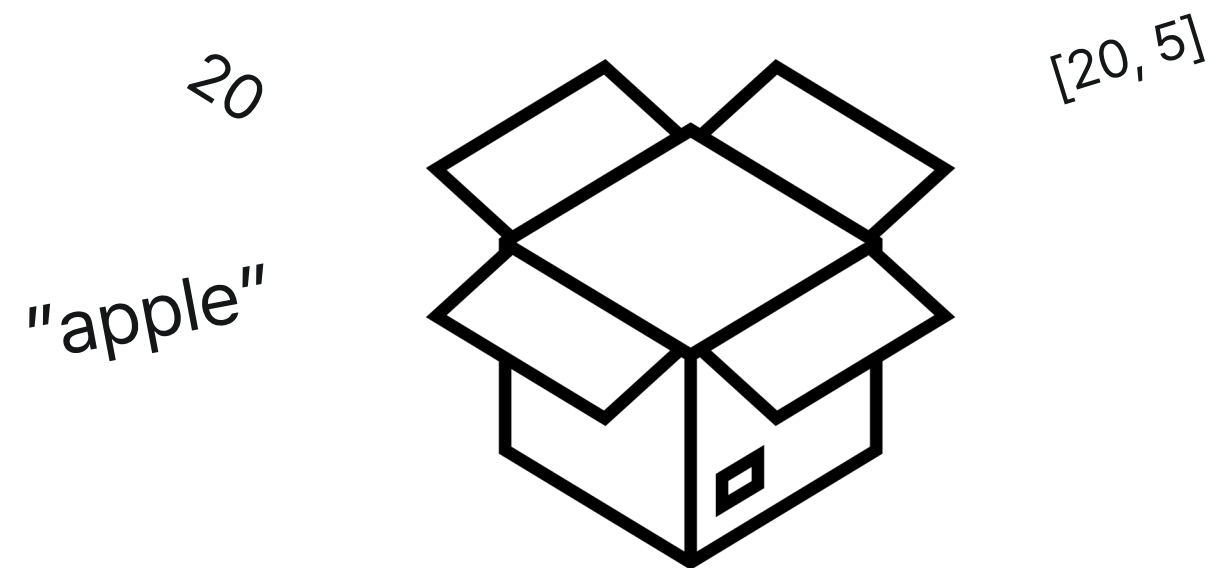
`print("출력할 내용")`



④ 변수

자료를 그릇에 담아서 보관, 사용하면 편리합니다.

이를 변수(variable)라고 합니다.





④ 변수에 저장 되어있는 값 출력하기

변수에 저장 되어있는 값을 출력하고자 한다면 어떠한 코드를 작성해야 할까요?

```
num = 12345  
print(num)
```

실행결과

12345

`print(값을 출력하고자 하는 변수명)`



④ 문자열 입력 받기

`input()` 함수를 활용하여 원하는 문자열을 입력 받을 수 있습니다.

```
user = input("원하는 문자열을 입력하세요.\n")    # "\n" 은 개행(줄 바꿈) 문자  
print(user)
```

실행결과

```
원하는 문자열을 입력하세요.  
안녕하세요. ← 입력한 문자열  
안녕하세요.
```

`input("입력 받을 때 출력할 내용")`



☑ 수학의 사칙 연산

+

더하기

-

빼기

×

곱하기

÷

나누기



☑ 파이썬에서의 사칙 연산자

+

더하기

-

빼기

*

곱하기

/

나누기



④ 변수를 활용한 사칙 연산

```
num1 = 3  
num2 = 5  
print(num1 + num2)  
print(num1 - num2)  
print(num1 * num2)  
print(num1 / num2)
```

실행결과

```
8  
-2  
15  
0.6
```



☑ 추가 연산자

//

몫

%

나머지

**

제곱



④ 변수를 활용한 몫, 나머지, 제곱 연산

```
num1 = 4  
num2 = 2  
print(num1 // num2)  
print(num1 % num2)  
print(num1 ** num2)
```

실행결과

```
2  
0  
16
```



④ 비교 연산

결과 값이 논리형 (True / False)으로 도출됩니다.

	설명
<	작은
<=	작거나 같은
>	큰
>=	크거나 같은
==	같은
!=	같지 않은



☑ 논리 연산

결과 값이 논리형 (True / False)으로 도출됩니다.

x or y

x,y 중 하나만
True이면 True

x and y

x,y 둘 다
True여야 True

not x

x가 True이면 False,
x가 False이면 True



📌 조건문

특정 조건에 따라서 실행되는 명령이 달라지는 구문

```
if 조건 1:  
    do A  
elif 조건 2:  
    do B  
elif 조건 3:  
    do C  
...  
else:  
    do X
```

조건 1 **True**

→ A 실행

조건1 **False and** 조건2 **True**

→ B 실행

조건1 **False and** 조건2 **False and** 조건3 **True**

→ C 실행

...

모든 조건이 **False**

→ X 실행



☑ if-else문

if의 조건이 True이면 if문 실행

if의 조건이 False이면 else문 실행

```
if 조건:
    <수행할 명령>
else:
    <수행할 명령>
```

```
eng = input()

if eng in ['a', 'e', 'i', 'o', 'u']:
    print("모음입니다.")
else:
    print("자음입니다.")
```



📌 if-elif-else문

조건1이 True이면 if문 실행

조건1이 False이고, 조건2가 True이면 elif문 실행

조건1이 False이고, 조건2가 False이면 else문 실행

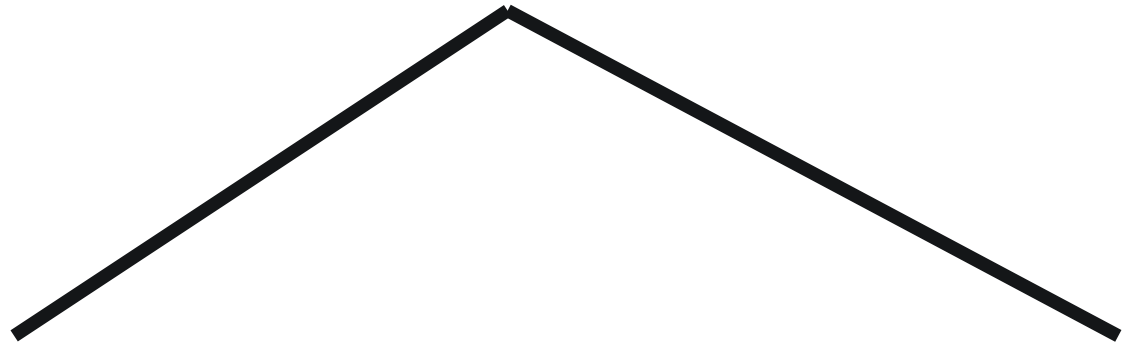
```
if 조건 1:  
    <수행할 명령>  
elif 조건 2:  
    <수행할 명령>  
else:  
    <수행할 명령>
```

```
score = int(input())  
  
if score >= 90:  
    print("A+ 입니다.")  
elif score >= 70:  
    print("B+ 입니다.")  
else:  
    print("C+ 입니다.")
```



④ 반복문

특정 조건이나 범위 내에서 어떠한 명령을 반복적으로 수행하는 것



for문

반복 횟수가
정해져 있을 때

while문

반복 조건이
정해져 있을 때



☑ for문

원소로 반복하는 방법으로 시퀀스의 원소를 하나 씩 변수에 넣어가면서 명령을 실행합니다.

```
for 변수 in 시퀀스:  
    <수행할 명령>
```

```
result = 0  
for i in [1, 2, 3]:  
    result = result + i
```



④ for-range

연속되는 숫자를 만들어주는 시퀀스 자료형인 range를 활용하면 특정 구간 / 횟수로 반복할 수 있습니다.

```
for 변수 in range(a,b):  
    <수행할 명령>
```

a ~ b-1 의 범위 실행

```
for 변수 in range(a):  
    <수행할 명령>
```

a 회 실행



🔍 while문

조건으로 반복하는 방법으로 조건이 True이면 명령을 실행합니다.

```
while 조건:  
    <수행할 명령>
```

```
i = 5  
while i>0:  
    print(i)  
    i = i - 1  
print("Launch!")
```



④ 반복문의 제어

특정 조건에서 반복문을 탈출할 수 있도록 해주는 break
특정 코드 실행을 건너뛸 수 있게 해주는 continue

```
i = 0
while True:
    print("knock")
    if i >= 3:
        break
    i = i + 1
```

break

```
i = 0
for i in range(0,4):
    if i == 3:
        continue
    print("knock")
```

continue



☑ 문자열과 리스트

문자열과 리스트는 순서가 있는 시퀀스 자료형입니다.

	설명	예시
int	정수형	1, 4, 10
float	실수형	1.23, 3.08
NoneType	None	None
str	문자열	"Hello, Python!"
list	리스트	[1,2,3,4]
bool	논리형	True, False
tuple	튜플	(1,2,3,4)
dict	딕셔너리	{'a':1, 'b':2}
set	세트	{1,3,5,7}



④ 시퀀스 자료형의 특징

특징	설명
원소 간의 순서가 존재	인덱싱 / 슬라이싱이 가능
멤버 조회	in 으로 시퀀스 안에 특정 원소가 있는지 확인 가능
길이 확인	len 으로 시퀀스 안에 원소가 몇 개인지 확인 가능
연결 연산	+ 연산자로 같은 시퀀스 두 개를 이어 붙이기 가능
반복 연산	* 연산자로 시퀀스 반복 가능



☑ 문자열

“ 또는 ’ 로 감싸져있는 시퀀스 자료형

```
a = "" # 빈 문자열 생성  
b = "Hello" # 문자열 생성  
  
print(a) #  
print(b) # Hello
```



☑ 문자열 활용

	설명
문자열.split(c)	c를 기준으로 문자열을 분할하여 리스트로 반환, 괄호가 비워져 있으면 공백을 기준으로 분할함
문자열.join(리스트)	문자열을 기준으로 리스트를 합쳐서 문자열로 반환, 괄호가 비워져 있으면 공백을 기준으로 합쳐서 반환



④ 리스트

`[]` 로 감싸져있는 시퀀스 자료형으로, 각 요소는 `,` 로 구분되어있다.

```
a = [] # 빈 리스트 생성
b = ['a', 'b', 'c'] # 요소를 담은 리스트 생성

print(a) # []
print(b) # ['a', 'b', 'c']
```



④ 리스트 활용

	설명
리스트.append(d)	자료 d를 리스트 마지막 원소 뒤에 추가, 오직 한 개의 자료만 넣을 수 있음
리스트.insert(i, d)	인덱스 i에 자료 d를 추가, 오직 한 개의 자료만 넣을 수 있음
리스트.remove(d)	처음 나오는 자료 d를 제거
리스트.sort()	리스트를 정렬, 숫자형은 오름차순으로 문자열은 사전순으로 정렬됨
리스트.pop()	인덱스 i의 원소를 제거하고 그 원소를 반환, 괄호가 비워져 있으면 마지막 원소가 제거됨



④ 딕셔너리

짝궁이 있는 자료형, `{key : value}` 의 형식으로 key를 통해 value를 알 수 있습니다.

	설명	예시
int	정수형	1, 4, 10
float	실수형	1.23, 3.08
NoneType	None	None
str	문자열	"Hello, Python!"
list	리스트	[1,2,3,4]
bool	논리형	True, False
tuple	튜플	(1,2,3,4)
dict	딕셔너리	{'a':1, 'b':2}
set	세트	{1,3,5,7}



④ 딕셔너리 활용

key는 변할 수 없는 자료형 즉, 리스트는 key가 될 수 없으며 튜플은 key가 될 수 있습니다.

	설명
딕셔너리[key]	자료를 꺼내거나, 추가, 수정
del 딕셔너리[key]	딕셔너리의 원소 삭제



④ 튜플

여러 개의 자료를 함께 담을 수 있는 시퀀스 자료형으로, **()** 로 묶어서 표현합니다.

순서가 있기 때문에 인덱싱, 슬라이싱이 가능합니다.

자료 추가, 삭제, 변경이 불가하여 한 번 만들어지면 값이 고정된다는 특징이 있습니다.

```
my_tuple = ('e', 'l', 'i', 'c', 'e')
```

```
print(my_tuple.append('!')) # 에러 발생
```

```
my_tuple[1] = 's' # 에러 발생
```